

Условие задачи

Задание кратко

Вам необходимо с нуля развернуть в Kubernetes простое веб-приложение и собрать вокруг него базовую инфраструктуру для публикации, мониторинга и сбора логов.

В результате должно работать следующее:

- веб-приложение развернуто в Kubernetes;
- доступ к приложению организован через Kubernetes Gateway API;
- Prometheus собирает метрики развернутой инфраструктуры;
- Fluentd или Filebeat собирает логи приложения;
- Решение должно поддерживать работу на Ubuntu 24.04;
- развертывание решения автоматизировано и может быть воспроизведено по вашей инструкции.

Итоговое решение должно быть полностью воспроизводимо на стороне проверяющих экспертов на основании материалов Git-репозитория и не должно требовать доступа к инфраструктуре участника или к специфическим коммерческим сервисам.

По итогам онлайн-этапа необходимо предоставить:

- Git-репозиторий с конфигурациями, кодом автоматизации и инструкцией по развертыванию;
- краткий паспорт решения, позволяющий экспертам быстро понять архитектуру.

Требования к обязательным частям решения

1. Kubernetes-окружение

Среду для разработки и тестирования участник выбирает самостоятельно. Kubernetes-кластер может быть развернут локально, на виртуальной машине, в собственной или облачной инфраструктуре. Предпочтительным решением считается kubectl от kubernetes.io (не привязанный к вендорам). Также допускается использование kind, minikube, k3d и других совместимых решений, но при прочих равных приоритет отдается решениям на kubectl.

Итоговое решение должно быть полностью воспроизводимо экспертами на основании материалов Git-репозитория и не должно требовать доступа к инфраструктуре участника.

Для выполнения обязательной части задания решение не должно зависеть от специфических коммерческих сервисов конкретного облачного провайдера.

Используемая версия Kubernetes должна быть указана в README.

Все дополнительные компоненты, необходимые для работы решения, должны быть перечислены в README, а способ их установки — описан или автоматизирован.

В README необходимо указать:

- используемую версию Kubernetes;
- способ создания кластера;
- операционную систему, на которой тестировалось решение.

Организаторы HE предоставляют участникам Kubernetes-стенд для работы над решением.

2. Демонстрационное веб-приложение

Необходимо развернуть в Kubernetes простое веб-приложение. Допускается использование любого простого open-source веб-приложения.

Например:

- Nginx;
- Angie;
- Apache HTTP Server;
- другое аналогичное open-source приложение.

Приложение должно:

- принимать HTTP-запросы;
- возвращать однозначно проверяемый ответ, например Hello World!;
- формировать access-логи HTTP-запросов, доступные для последующего сбора Fluentd или Filebeat.

Приложение должно позволить однозначно проверить успешность HTTP-запроса.

Используемый container image должен быть публично доступен либо участник должен предоставить в Git-репозитории все необходимые материалы для его сборки.

Разработка собственного приложения не является целью задания и не дает дополнительных баллов.

3. Gateway API

Доступ к приложению необходимо организовать через Kubernetes Gateway API.

Минимально должны быть настроены:

- реализация Gateway API / Gateway Controller;
- GatewayClass (если он требуется выбранной реализацией);
- Gateway;
- HTTPRoute, направляющий HTTP-трафик к Kubernetes Service приложения;
- доступность приложения через настроенный Gateway API.

Участник самостоятельно выбирает open-source реализацию Gateway API.

В README необходимо указать:

- название реализации;
- ее версию;
- используемые ресурсы Gateway API.

Участник должен предоставить способ проверки доступности приложения через Gateway API, например команду curl, возвращающую ожидаемый ответ приложения.

4. Мониторинг

Необходимо настроить мониторинг с использованием Prometheus.

Prometheus должен получать метрики как минимум от одного компонента развернутого решения.

Вам необходимо:

- настроить сбор метрик;
- убедиться, что соответствующий target доступен Prometheus;
- продемонстрировать получение метрик с помощью Prometheus query;
- описать в README, какие метрики собираются и как эксперт может проверить их получение.

Сбор дополнительных метрик приложения и инфраструктуры является дополнительным преимуществом.

5. Логирование

Необходимо настроить сбор логов демонстрационного приложения с использованием Fluentd или Filebeat.

Вам необходимо:

- обеспечить сбор access/error-логов приложения;
- настроить их передачу в выбранное хранилище или точку назначения;
- продемонстрировать, что после обращения к приложению соответствующая запись появляется в собранных логах;
- описать в README, какие логи собираются, куда они поступают и как проверяющий эксперт может проверить их получение.

Способ хранения и просмотра логов вы выбираете самостоятельно.

6. Поддержка операционной системы

Необходимо подтвердить работоспособность решения на следующей операционной системе:

- Ubuntu 24.04.

Данная операционная система свободно доступна для скачивания и не ограничена к использованию.

7. Автоматизация развертывания

Решение должно быть воспроизводимым.

После выполнения описанной вами последовательности действий проверяющий эксперт должен иметь возможность получить работоспособное решение без ручного создания и редактирования Kubernetes-ресурсов.

Повторный запуск процесса развертывания не должен приводить систему в некорректное состояние.

Разрешается использовать:

- Kubernetes manifests;
- Helm;
- Kustomize;
- Terraform;
- Ansible;
- shell-скрипты;
- Python-скрипты;
- иные open-source инструменты.

Ручная настройка каждого Kubernetes-ресурса через графический интерфейс не считается полноценной автоматизацией.

Желательно свести развертывание к минимальному количеству понятных команд.

8. Документация

В Git-репозитории должна находиться инструкция, позволяющая понять архитектуру решения, развернуть его и проверить работоспособность основных компонентов.

Возможности для улучшения решения:

Дополнительные возможности не являются обязательными для выполнения задания, но будут учитываться при оценке решений на прохождение в финал. Обязательно опишите, какие дополнительные улучшения вы реализовали в своем решении в README и паспорте проекта. Список снизу – пример, вы можете добавлять улучшения, которые в нем не указаны.

Демонстрация в решении расширенных возможностей Gateway API, например:

- несколько маршрутов;
- маршрутизация по hostname или path;
- несколько backend;

- TLS;
- traffic splitting;
- другие возможности Gateway API.

CI/CD

Можно реализовать pipeline, который автоматизирует, например:

- проверку конфигураций;
- сборку;
- развертывание;
- smoke-тестирование;
- другие этапы жизненного цикла решения.

Расширенный мониторинг и логирование, например:

- HTTP-метрики;
- количество запросов;
- HTTP-коды ответов;
- latency;
- CPU/RAM;
- дополнительные dashboards;
- централизованное хранение и поиск логов.

Дополнительные практики надежности и безопасности

Можно реализовать дополнительные механизмы, которые считаете полезными для эксплуатации решения.

Важно: дополнительная функциональность учитывается только при наличии работоспособного базового решения.

Предлагаемый роадмап

Рекомендуемый путь выполнения задания:

Шаг 1. Подготовьте Kubernetes-окружение

Выберите удобный вариант Kubernetes и убедитесь, что кластер работает.

Например, можно использовать:

- kubeadm (приоритет);
- kind;
- minikube;
- k3d;
- собственный Kubernetes-кластер;
- другие доступные решения.

Предпочтительным решением является использование kubeadm от kubernetes.io.

Шаг 2. Разверните веб-приложение

Создайте необходимые Kubernetes-ресурсы и убедитесь, что приложение работает внутри кластера.

Шаг 3. Настройте Gateway API

Выберите open-source реализацию Gateway API.

Создайте необходимые ресурсы и организуйте через них доступ к приложению.

Проверьте, что пользовательский HTTP-запрос действительно достигает приложения.

Шаг 4. Подключите мониторинг

Разверните или подключите Prometheus.

Настройте получение метрик и убедитесь, что Prometheus получает данные из вашей инфраструктуры.

Шаг 5. Настройте сбор логов

Подключите Fluentd или Filebeat.

Убедитесь, что после обращения к веб-приложению соответствующая запись появляется в выбранной системе сбора логов.

Шаг 6. Автоматизируйте развертывание

Соберите необходимые manifests, Helm/Kustomize-конфигурации, Terraform/Ansible или скрипты таким образом, чтобы решение можно было воспроизвести по инструкции.

Проверьте повторный запуск развертывания.

Шаг 7. Оформите решение

Подготовьте Git-репозиторий и паспорт решения.

Убедитесь, что проверяющий эксперт сможет быстро понять:

- как устроено решение;
- как его запустить;
- как проверить Gateway API;
- как проверить мониторинг;
- как проверить логирование.

Критерии оценки — 100 баллов

1. Работоспособность Kubernetes-решения и Gateway API — 30 баллов

Оценивается:

- корректность развертывания;
- работоспособность приложения;
- корректное использование Gateway API;
- работоспособность маршрутизации;
- логичность архитектуры;
- использование kubernetes.

2. Воспроизводимость и автоматизация — 25 баллов

Оценивается:

- возможность развернуть решение по инструкции;
- степень автоматизации;
- количество необходимых ручных действий;
- повторяемость результата;
- идемпотентность;
- управление конфигурацией и зависимостями.

3. Мониторинг и логирование — 20 баллов

Оценивается:

- корректность работы Prometheus;
- наличие реально собираемых метрик;
- корректность работы Fluentd/Filebeat;
- наличие реально собираемых логов;
- качество выбранного подхода к observability.

4. Качество технической реализации — 15 баллов

Оценивается:

- структура Kubernetes-конфигураций;
- качество кода и скриптов;
- разумность технических решений;
- базовые практики надежности и безопасности;
- отсутствие чувствительных данных в репозитории.

5. Документация и дополнительные возможности — 10 баллов

Оценивается:

- качество README;
- понятность архитектуры;
- простота воспроизведения;
- описание ограничений решения;
- CI/CD;
- дополнительные возможности Gateway API;
- другие обоснованные улучшения.

Дополнительная функциональность не компенсирует отсутствие или неработоспособность обязательных компонентов.

Что необходимо сдать

Вашим решением на онлайн-этапе является ZIP- или RAR-архив, содержащий два обязательных артефакта:

1. ТХТ-файл, содержащий ссылку на ваш Git-репозиторий

В репозитории должно находиться полное техническое решение и документация.

2. Паспорт решения

Краткий документ, который позволит проверяющим экспертам понять основные особенности вашего проекта.

Подробные требования к каждому артефакту приведены ниже в разделе “Требования к сдаче решения”.

Отсутствие обязательной части решения может стать основанием для недопуска работы к полной экспертной оценке.

Условия прохода в финал:

В очный финал (Москва) приглашаются **топ-10** участников по итогам онлайн-этапа. Отбор осуществляется **по оценке жюри** на основе критериев №1–5, указанных выше.

Требования к сдаче решения

Общие требования

Ваше решение должно быть сдано в виде архива, содержащего следующие элементы:

- TXT-файл с ссылкой на Git-репозиторий;
- Паспорт проекта.

Технические требования к архиву:

- Формат: ZIP или RAR;
- Объем: не более 18 МБ;
- Название архива: Ваша фамилия (как указывали при регистрации);
- Названия файлов внутри архива: "Ссылка.txt" и "Паспорт.doc" / "Паспорт.pdf".

Требования к каждой части решения и Git-репозиторию указаны ниже.

Прием решений производится до 4 октября 23:59, просим присылать решения заблаговременно.

Чтобы сдать решение нажмите на кнопку "Загрузить" на той же странице, на которой скачали задание, и прикрепите архив.

В случае, если вам необходимо заменить уже сданное решение, нажмите на кнопку "Загрузить другой файл" рядом с плашкой "Решение загружено". В таком случае новый архив заменит собой предыдущий.

Отсутствие обязательной части решения может стать основанием для недопуска работы к полной экспертной оценке.

Требования к Git-репозиторию

Технические требования:

Репозиторий может быть размещен на GitHub, GitLab или другом публично доступном Git-хостинге.

ВАЖНО: Git-репозиторий должен быть публичным и доступным без авторизации на протяжении всего периода проверки решений. Проверяющие эксперты должны иметь возможность клонировать репозиторий стандартными средствами Git. Перед сдачей решения убедитесь, что ссылка доступна и команда `git clone <URL>` выполняется без запроса дополнительных прав доступа.

Репозиторий должен оставаться публично доступным до публикации списка финалистов.

Решение должно располагаться в основной (main) ветке репозитория.

Внесение изменений в ветку репозитория после 4 октября 23:59 **КАТЕГОРИЧЕСКИ ЗАПРЕЩЕНО** и может привести к дисквалификации участника.

В репозитории должны находиться:

1. Исходные файлы для развертывания

Все Kubernetes manifests, Helm charts, Kustomize-конфигурации, Terraform/Ansible-файлы, скрипты и другие материалы, необходимые для воспроизведения решения.

2. Конфигурация Gateway API

В репозитории должны находиться все необходимые файлы для развертывания и настройки Gateway API.

3. Конфигурация мониторинга

Материалы, необходимые для настройки Prometheus и сбора заявленных метрик.

4. Конфигурация логирования

Конфигурация Fluentd или Filebeat и иных компонентов, необходимых для реализации выбранной схемы работы с логами.

5. README.md

README должен позволить проверяющему эксперту понять и воспроизвести решение.

README обязательно должен содержать:

- краткое описание решения;
- архитектуру;

- список использованных технологий и их версий;
- использованную версию Kubernetes;
- выбранную реализацию Gateway API;
- требования к среде;
- пошаговую инструкцию по развертыванию;
- команду или последовательность команд для запуска;
- инструкцию по проверке доступности приложения;
- инструкцию по проверке мониторинга;
- инструкцию по проверке логирования;
- описание дополнительных возможностей;
- известные ограничения решения.

Желательно

Сведите первоначальное развертывание к одной или нескольким явно описанным командам.

Например:

```
make deploy
```

или

```
./deploy.sh
```

или аналогичному сценарию.

Требования к безопасности

Не размещайте в репозитории:

- реальные пароли;
- API-токены;
- приватные SSH-ключи;
- персональные данные;
- другие секреты.

Если для работы решения необходимы секреты, предусмотрите безопасный способ их передачи через переменные окружения, Secret или шаблон конфигурации.

Формат сдачи

В архив с решением приложите TXT-файл, который содержит ТОЛЬКО ссылку на актуальную ветку Git-репозитория. TXT-файл должен называться "Ссылка.txt".

Требования к паспорту проекта

Паспорт решения используется экспертами для скрининга работ.

Его задача — позволить за несколько минут понять:

- что было реализовано;
- как устроено решение;
- какие обязательные компоненты работают;
- какие дополнительные возможности реализованы;
- стоит ли решение направлять на углубленную техническую проверку.

Формат

- PDF или DOC/DOCX;
- не более 4 страниц;
- размер файла — не более 15 МБ.

Не рекомендуется превращать паспорт в полноценную презентацию или техническую документацию. Подробная документация должна находиться в Git-репозитории.

Структура:

= Страница 1. Архитектура и состав решения

Укажите:

- используемую версию Kubernetes;
- способ развертывания Kubernetes;
- выбранную реализацию Gateway API;
- используемые инструменты автоматизации;
- используемый инструмент логирования — Fluentd или Filebeat;
- способ развертывания Prometheus;
- операционную систему, на которой тестировалось решение.

Добавьте простую архитектурную схему.

На схеме рекомендуется показать:

Пользователь > Gateway API > приложение

а также компоненты мониторинга и логирования.

= Страница 2. Реализованный функционал

Укажите:

- Для каждого обязательного пункта решения:
 - Как технически был реализован
 - Обоснование выбора\реализации
 - Как проверяющий эксперт может проверить работоспособность
- Для каждого дополнительного улучшения решения:
 - Что было реализовано
 - Как технически было реализовано
 - Обоснование выбора\реализации
 - Как проверяющий эксперт может проверить работоспособность

При необходимости на этот раздел паспорта можно выделить дополнительную страницу.

= Страница 3. Ревью работы и потенциальное масштабирование.

Кратко (не более 3 предложений на пункт) укажите:

- Главная особенность вашего решения:
что вы считаете наиболее сильной технической стороной проекта.
- Какое принятое решение было самым сложным во время работы над проектом:
между какими вариантами вы выбирали и на основании чего приняли решение

Предложения по дальнейшему развитию решения:

- Списание укажите, что можно добавить в решение и что для этой реализации необходимо (например, платные сервисы или специфическая инфраструктура). При возможности рекомендуется учитывать телеком-специфику.

Важно

Паспорт решения не заменяет репозиторий.

Все заявленные в паспорте возможности должны подтверждаться содержимым Git-репозитория и могут быть проверены экспертами.

Если заявленная функциональность отсутствует в репозитории или не может быть воспроизведена, она не учитывается при выставлении итоговой оценки.